Optimizing Attention-Based Flower Classification for Edge TPU: A Performance Analysis from Training to Deployment

Ivan Morrow

Electrical and Computer Engineering
Colorado State University

Fort Collins, CO
Ivan.Morrow@colostate.edu

***Abstract*— In this paper we describe how to fine-tune and quantize a pre-trained model designed for image classification, with a focus on integrating efficient data pipelines and inference. The model will be deployed to a Coral Edge TPU device and will showcase optimizations targeted for the TPU platform.**

Keywords—image classification, TPU, parallel processing, MobileNet

I. Introduction

In this project, we demonstrate the end-to-end process of deploying an efficient image classification system on edge hardware. We fine-tune MobileNetV2, a model architecture specifically designed for resource-constrained devices, using the flowers dataset available in TensorFlow. This comprehensive dataset contains thousands of images of different flowers. Our pipeline encompasses data preprocessing, model training, and optimization through quantization to ensure compatibility with the Edge TPU architecture. The Edge TPU, a specialized accelerator designed for efficient on-device machine learning inference, serves as our target deployment platform. We evaluate the deployed model's effectiveness through extensive testing, measuring critical metrics including prediction accuracy, inference latency, and overall performance. This work illustrates the practical implementation of computer vision systems on edge devices.

II. Problem Statement

The integration of machine learning models into edge devices presents unique challenges in optimizing performance while managing computational constraints. This project addresses the specific challenge of implementing efficient image classification on edge devices through comprehensive performance analysis and optimization. While cloud-based solutions offer substantial computational resources, edge deployment demands careful consideration of processing efficiency and hardware utilization. By leveraging the Coral Dev Board's Edge TPU architecture and the MobileNetV2 model architecture, we demonstrate a systematic approach to performance optimization that achieves efficient on-device inference while maintaining classification accuracy.

The significance of this implementation extends beyond technical demonstration, establishing a framework for understanding and optimizing machine learning deployments on edge devices. Our comprehensive benchmarking methodology provides insights into the relationships between model architecture, data pipeline efficiency, and hardware utilization. Through detailed analysis of parallel processing implementations and TPU-specific optimizations, we establish best practices for deploying sophisticated machine learning applications on resource-constrained edge devices while maintaining real-time performance requirements.

This study specifically focuses on optimizing edge computing performance through:

1. Implementation of efficient parallel processing and data pipeline optimizations
2. Development of comprehensive performance benchmarking methodologies
3. Analysis of TPU-specific optimizations and their impact on inference speed
4. Quantitative evaluation of warmup effects and hardware utilization metrics

This approach advances the field by providing a detailed framework for performance analysis and optimization of edge-deployed machine learning models. By examining the interplay between data preprocessing, model architecture, and hardware utilization, we establish a comprehensive understanding of performance optimization in edge computing environments. The findings provide valuable insights for developers and researchers working to deploy efficient machine learning solutions on edge devices.

A. Dataset

The TensorFlow Flowers dataset serves as our training dataset, containing 3,670 images across five flower categories: daisy, dandelion, roses, sunflowers, and tulips. We utilize this dataset exclusively in our Google Colab training environment, leveraging its well-structured organization and TensorFlow's optimized data handling capabilities to train our classification model effectively.

The dataset's structure and characteristics align well with our performance optimization objectives. Each image class contains between 500 to 1,000 examples, providing sufficient data for both training and evaluation while maintaining a manageable size for edge deployment scenarios. The images come in various resolutions and aspect ratios, requiring our preprocessing pipeline to handle diverse input formats efficiently. This variety helps validate our pipeline's robustness and optimization strategies under different computational loads.

For our implementation, we leverage TensorFlow's built-in data loading mechanisms through the TensorFlow datasets (TFDS) API, which provides optimized data handling capabilities. The dataset's organization enables straightforward splitting into training and validation sets, with our implementation using an 80-20 split ratio. This division ensures adequate data for both model training and performance evaluation while maintaining statistical significance in our benchmarking results.

The dataset's characteristics make it particularly suitable for evaluating our parallel processing and TPU optimization strategies. The varying image sizes and complexities create different computational demands, allowing us to assess the effectiveness of our optimizations across diverse workloads. This variation helps demonstrate the real-world applicability of our performance optimization techniques and their impact on inference speed and efficiency.

B. Problems

During the project's development, I encountered challenges that led to a slight redirection of the project. The initial project scope aimed to perform real-time object detection on the Edge TPU incorporating comprehensive benchmarking, parallel processing, and TPU optimizations specific to detection tasks. However, there were unforeseen issues I ran into fine-tuning an object detection model that required a shift in direction to complete the project on time.

To ensure the delivery of meaningful results and maintain the focus on performance optimization, I shifted the implementation to address image classification tasks. This adjustment allows me to demonstrate the Edge TPU's capabilities through a focused analysis of inference performance on deployed test images. The revised approach maintains the core technical objectives of the study - evaluating model performance, implementing parallel processing optimizations, and analyzing TPU-specific enhancements.

III. IMPLEMENTATION

1) Data Pipeline

A data and training pipeline has been created from scratch to help fine-tune the pre-trained MobileNetV2 model. Before I fine-tune the model, I convert the data to TensorFlow Record (TFRecord) format. TFRecord is a binary file format that is optimized for TensorFlow specifically. By converting our training images to TFRecords, we create a more efficient training pipeline as binary format is much more efficient to parse than image files. This also allows for faster reading of the data. Internally, TFRecords use Protocol Buffers (Protobuf) to serialize/deserialize the data and store them in byte format. Protobufs are a schema based way to define data formats that producers/clients can use to serialize/deserialize byte buffers they receive into human readable formats. When a service receives a raw byte buffer, it can use protobufs to deserialize that byte buffer into a schema that is defined by the protobuf. Protobufs are language and platform agnostic. Protobufs are used extensively outside of Google and we use them as our main method of transmitting data between components of our system where I work at Belvedere Trading.

In the data pipeline, we convert all of the training and validation images into shared TFRecord files. We are able to serialize all of this data across multiple files while maintaining the data integrity. TFRecords also allow for streaming of data rather than loading each training sample into memory one at a time which increases the throughput of the training. Lastly, TFRecords are compatible with TPUs which is what was used to fine-tune this model. All of this was implemented in a Google Colab notebook using the TPU runtime for execution.

TFRecords are really useful when you have very large datasets. Sharding allows for more parallel processing and better memory management. With very large datasets, converting to TFRecord format will cut down on the size of the dataset alone, but it is possible that a single TFRecord format could still be very large and cumbersome for a training pipeline to handle. To help with that, you can further reduce the size required in memory to store training data by sharding the single TFRecord file into multiple. This can allow for more efficient distributed training, parallel processing as multiple tfrecord files can be read at once, and it prevents loading the entire dataset into memory. All of that is to say, TFRecords are very performant and are treated as first-class citizens in TensorFlow. With them, we can create much more efficient pipelines. The real benefits of TFRecords are on display when you have an extremely large dataset and you are able to utilizing data streaming to pass data to the model while its training. Our dataset used in this project is too small to see the benefits from data streaming, so we do not implement this but that is something that could be explored in further iterations.

Additionally, the data pipeline utilizes data augmentation to create synthetic training data by slightly altering the existing training images. To help do this efficiently, we utilize some TensorFlow and TPU optimizations. We have a function augment(), that just takes an image and label as arguments, flips the image and adjusts the brightness. However, we are able to utilize parallel processing with our TPUs to run this operation by calling the map() function on the TensorFlow dataset API. The tf.data.Dataset.map function in TensorFlow is used to apply a transformation to each element of a dataset. This function can be executed in parallel, the API has an optional argument Num_parallel_calls that controls the level of parallelism. In this case, it determines how many transformations can be executed in parallel. For ease of use, we set num_parallel_calls=AUTOTUNE, which allows TensorFlow to determine how many operations to execute in parallel depending on the hardware available. TensorFlow will leverage the multiple TPU cores available in the Colab environment and execute the augment() function on different parts of the dataset in parallel. With the AUTOTUNE configuration, TensorFlow will dynamically adjust the number of parallel calls based on the data and available resources, aiming to keep the TPU cores busy without causing any bottlenecks.

Identify applicable funding agency here. If none, delete this text box.

2) Model Architecture

Once the training data has been preprocessed and prepared, we must re-architect the original MobileNetV2 model to be fine-tuned on it. The retrained model needs to be adapted to the TensorFlow flowers specific class categories and the model must output predictions for our specific flower dataset classes, of which there are 5.

Our implementation leverages MobileNetV2, a model architecture specifically designed for efficient deployment on mobile and edge devices. The model was initially trained on the ImageNet dataset, providing it with robust feature extraction capabilities across a wide range of image classifications. For our specific flower classification task, we employ transfer learning methodologies to adapt this pre-trained architecture to our requirements.

The model adaptation process begins by removing the original classification layers from MobileNetV2 while preserving the base model's weights. This approach capitalizes on the model's pre-learned feature extraction capabilities, particularly in the lower convolutional layers, which have developed general pattern recognition abilities applicable across various image classification tasks. The preserved base weights provide a strong foundation for our specific classification requirements.

We then augment the model with a new classification head specifically designed for our five-class flower classification task. When designing the new classification head, optimization for TPUs was taken into account. In addition to the classification head, there are two added layers to the base model that are worth describing. The output from the base MobileNetV2 model is passed to a GlobalPooling2d layer. The output from the base model at this point will contain a feature map with spatial information about the different features detected in an image. This added layer takes that feature map and converts our grid of features into a single vector. This is to ensure compatibility with our classification head, which expects an input of a single vector of numbers. So, our 2D matrix for each feature with spatial information is converted to a single vector, with an entry for each unique feature in the image. At this point, all features are weighted equally.

The GlobalPooling2d layer feeds into an Attention layer that has been optimized for TPU execution, whose job is to identify what features are important for each flower type. Different flowers can be distinguished by very specific features - the arrangement of petals, the shape of the center, the texture of leaves. Not all features are equally important for identifying every type of flower. The goal of the attention layer is to created weighted values to determine which features are most important for our task of flower classification. These weights will be multiplied by the output of the feature extraction layer (GlobalPooling2d layer) to come up with new weighted feature values. The attention layer employs a three-step process to identify and emphasize important features in the model. Initially, it transforms the input features into a 512-dimensional space, leveraging the TPU's optimized architecture for matrix operations with power-of-2 dimensions. The next layer converts this 512 dimensional space to attention weights with values between 0 and 1, creating an importance score for each feature. A value closer to 1 means that this particular feature is important. Lastly, the attention weights are applied to the features through element-wise multiplication. Through the training process and error backpropagation, the attention layer progressively refines these weights, learning to distinguish crucial features for flower classification. Finally, the attention weighted feature values are passed to the classification layer, which feeds into the output layer.

This architecture allows the model to learn what features are most important to the image classification and emphasize these features when performing inference. This results in a high performing model.

3) Fine-Tuning and Transfer Learning

During the fine-tuning process, we implement a careful training strategy that balances the preservation of the model's general feature extraction capabilities with adaptation to our specific classification task. This approach minimizes the risk of overfitting while ensuring the model develops strong predictive capabilities for flower classification. The resulting architecture maintains the computational efficiency necessary for edge deployment while achieving high accuracy in flower classification tasks.

In the Colab training environment, we have initialized a TPUStrategy class that configures the available TPUs for computation. Because of this, there is nothing we need to explicitly do in our train function to utilize them. That configuration will automatically distribute the training process across available TPU cores, enabling parallel execution and faster training. We do optimize our batch size during training for the number of TPU cores available. TPUs excel with larger batch sizes due to their parallel processing capabilities. Our training process adjusts the batch size based on the number of TPU replicas, ensuring efficient utilization of TPU resources.

In our training pipeline, we initially convert our data into sharded TFRecord files before transforming them into TensorFlow Datasets for training. While this approach provides structure and organization to our data pipeline, it doesn't fully leverage the streaming capabilities that TFRecords can offer. For scenarios involving larger datasets or distributed training environments, implementing direct TFRecord streaming could provide significant performance benefits. To fully utilize TFRecord streaming capabilities, one could implement a data pipeline that reads directly from TFRecord files during training, rather than converting to intermediate Dataset objects. This approach would enable true streaming benefits: data would be loaded in batches as needed, reducing memory overhead and potentially improving training speed through more efficient data access patterns. Such an implementation would be especially beneficial for production environments where data efficiency and scalability are crucial considerations.

In our current implementation, while we maintain the organized structure of TFRecords, we focus on optimizing the training process through our TPU-specific configurations and attention mechanism architecture. For future iterations or larger-scale deployments, implementing direct TFRecord streaming could provide an additional avenue for performance optimization.

Finally, we can train the model. Training the model at this point is not very different from training a regular TensorFlow model. When we imported the MobileNetV2 base model, we set it with trainable=False so that its weights will not be

updated by TensorFlow. So, now after we have added the GlobalPooling2d layer, attention mechanism, and classification layer to it we can call model.fit() and the added layers will be trained on the given dataset. This is where we pass in our flowers dataset and our model will learn to identify the different flower classes.

After training, the model in its current state is incompatible for most edge devices, including the Edge TPU. Most devices are able to run TensorFlow models converted to tflite models, but the Edge TPU requires an additional step that we will describe in the next section.

4) Edge TPU Deployment

The purpose of this model is to be deployed to an Edge TPU device which has very specific requirements for models to run effectively on the platform. In order to provide high-speed neural network performance with a low-power cost the Edge TPU supports a specific set of neural network operations and architectures. It supports only TesnorFlow Lite models that are fully 8-bit quantized and then compiled specifically for the Edge TPU. The Edge TPU has a specific Edge TPU Compiler that will convert a traditional TensorFlow Lite model to one that is compatible with the Edge TPU. Before we dive into what the compiler does, lets talk about our quantization process, as that is not handled by the compiler itself but it does expect any model it compiles is already quantized.

We quantize the model to use 8-bit INTs instead of 32-bit Floating-Point INTs in its weights and operations, which is the default format for TensorFlow models. The Edge TPU device only supports 8-bit int operations because it is a resource constrained environment and less precision generally requires less resources (memory, computational capacity, power). Now, you could also accomplish this with quantization-aware training so that the resulting TensorFlow model is already quantized, but that takes longer and is slightly more complex.

After quantization, we can convert the model to a tflite model and store it in a tflite file. This file is what we will pass into the Edge TPU compiler. The Edge TPU compiler is a command line interface (CLI) tool created by Google for the purpose of converting TFlite models to a format that is compatible for efficient performance on the Edge TPU device.

The Edge TPU compiler does the following:

- Performs a verification that the model meets all requirements for the Edge TPU. Mainly that it only utilizes INT8/UINT8 Ops.
- Reorganizes the models weights and operations for Edge TPU's memory architecture.
- Translates TFLite operations into Edge TPU machine code and maps operations to the Edge TPU custom instruction set.
- Partitions code into two sets: Edge TPU compatible operations that will run on the TPU and incompatible ops that will be passed to run on the CPU. It will also generate any code required to coordinate between TPU and CPU.

Without this compilation process, the tflite model would not be compatible with the Edge TPU. After a successful compilation, a new model will be stored in a tflite file. Although it is still technically a tflite file, it has been changed significantly to run efficiently on the Edge TPU device. If the compiler deems that your model does not meet all of the requirements to execute on the TPU, it will still compile but all operations after the first encountered unsupported operation will be moved to execute on the CPU instead of the TPU.

5) Further Optimizations

Throughout the training pipeline and during inference on the device, there are a few more optimizations we make use of to optimize the performance. One of the most significant optimizations is our utilization of TensorFlow graphs through the @tf.function decorator. This optimization transforms Python functions into TensorFlow graphs, providing substantial performance improvements over traditional Python execution.

In traditional Python, code is executed sequentially, with each operation being interpreted and executed one at a time. This approach, while flexible and developer-friendly, introduces significant overhead in computational workflows. TensorFlow graphs, by contrast, define the entire computation as a directed graph of operations that can be optimized and executed efficiently.

When we apply the @tf.function decorator to our data preprocessing and augmentation functions, TensorFlow converts these Python functions into graph operations. This conversion enables several key optimizations. First, the graph representation eliminates Python's interpretation overhead, as the operations are compiled into optimized code that can be executed directly by TensorFlow's runtime. In traditional Python, the interpreter will iterate over the function line by line and interpret each operation at runtime. This is slow. By utilizing TensorFlow graphs, the execution is optimized behind the scenes and can run much faster than python code. Second, the graph structure allows TensorFlow to analyze the entire computation and apply optimizations such as operation fusion, where multiple operations can be combined into more efficient implementations. Third, graphs enable better parallelization of operations, as TensorFlow can identify independent computations that can be executed simultaneously.

In our implementation, we particularly benefit from graph optimization in our data augmentation pipeline. Operations such as random flipping and brightness adjustments, when converted to graph operations, can be executed more efficiently and in parallel. This optimization becomes especially important when processing large batches of images, as the performance gains compound with the scale of data processing.

These optimizations demonstrate how careful consideration of computational models and leveraging framework-specific features can lead to significant performance improvements in machine learning workflows. By utilizing TensorFlow graphs, we achieve faster execution times and better resource utilization, contributing to the overall efficiency of our training and inference pipeline.

6) System-Wide Optimization Strategies for TPU-Based Training and Inference

There were a lot of different optimization methodologies used throughout this project, both on the training and inference side in an attempt to utilize the available hardware efficiently. There is not doubt further optimizations to be made, but I will summarize all of the optimizations we have done so far.

First and foremost, we correctly configure the available TPUs at the start of our training pipeline to work hand in hand with TensorFlow. Simply connecting to the TPU runtime in Colab does not actually utilize them effectively. To do that, you must use the TPUClusterResolver() API from TensorFlow. You can use this to initialize TensorFlow with the TPUs in your environment and the TPUStrategy() API will distribute any work TensorFlow does among the available TPU cores for parallel processing. TensorFlow will do this automatically with this configuration.

Next, we configured the training batch sizes to dynamically change given the number of cores available. To do this, we simply multiplied the given batch_size by the number of available TPU cores. This will allow for distributed, parallel processing of training batches evenly among all of the TPU cores ensuring efficient memory utilization. TPUs excel with larger batch sizes and this ensures that the TPUs do not sit idle during training.

Next, we made heavy use of parallel features in the TensorFlow API when loading the data and converting it from TFRecord format. By leveraging the tf.data.AUTOTUNE API, we are able to indicate to TensorFlow that a specific operation should be done in parallel. TensorFlow dynamically adjusts the number of parallel threads for data processing based on the available TPU resources and the runtime characteristics of our input pipeline. This helps optimize the performance of data pipelines without requiring manual tuning.

We utilize the @tf.function decorator to convert specific functions from Python function to TensorFlow computation al graphs. TensorFlow can optimize graphs for far greater performance than a traditional Python function. TensorFlow graphs are portable, can operate independent operations in parallel, and allow TensorFlow to perform static analysis and apply optimizations like operation fusion and kernel optimizations. All of which lead to faster and more efficient execution. We use the @tf.function decorator of a few key functions throughout the training process.

In the design of our model architecture, we incorporated layer sizes that TPUs are built to operate efficiently on (powers of 2). We also only include operations that run very efficiently on TPUs (relu, softmax, sigmoid, element-wise multiplication).

Lastly, we trained the model utilizing the dynamically calculated batch_size that scales with the number of TPU cores and run model.fit() in an environment with the TensorFlow API configured to utilize the TPUs. This ensures the operations are run on the TPUs in the notebook.

IV. Preliminary Results

1) Data Pipeline

The training and data pipeline was implemented in a Google Colab environment utilizing Google's Tensor Processing Units (TPUs). TPUs are Google's own custom-designed AI accelerator hardware, they are not GPUs - they are Application Specific Integrated Circuits (ASICs). In the training and data pipeline of fine-tuning the MobileNetV2 model, we have tried to efficiently utilize as much of the available TPU hardware as possible through Colab. Utilizing TPUs effectively is more involved than simply selecting the TPU runtime in your notebook environment. To offer comparative analysis, we have also prepared a baseline training pipeline that does not utilize the TPUs as effectively as possible so that we can see the difference. In our pipeline, we distribute the computation across multiple TPU cores allowing for parallel processing. In our notebook, we have access to 8 TPU cores to utilize.

For context, both pipelines perform transfer learning of the MobileNetV2 model on the tf_flowers dataset for only 2 epochs. The benchmark analysis measures a few different metrics of each training pipeline, mainly the following: elapsed training time, throughput (images per second), model parameters, model accuracy, and loss.

	No TPU	TPU Optimized
Total Training Time	78.43 S	39.29 S
Total Model Parameters	2,422,597	4,362,053
Training Accuracy	88.9%	88.7%
Validation Accuracy	95%	72.4%
Model Size		

Clearly, the TPU shines when it is given the opportunity to process large amounts of data. The total training time is cut in half when optimized properly and the throughput of the pipeline increases tremendously. It is a little surprising to see the model parameters increase by >2 million in the TPU optimized training pipeline. That warrants further investigation. Additionally, it is interesting that after 2 epochs, the baseline training pipeline has a higher validation accuracy than the TPU optimized one. I am curious to see if this is the case as we increase the epochs.

2) Edge TPU Inference

For on-device inference, we implemented a benchmarking script to evaluate the model's performance on the Edge TPU. To ensure accurate results, we utilized a process known as "warming up" the TPU. This technique addresses the latency that occurs when the device performs inference from an idle state. By passing dummy data through the model prior to actual inference, the device is "warmed up," allowing it to reach optimal performance more quickly. In our benchmark, we conducted 1,000 iterations, with each iteration randomly selecting one of 10 pre-stored images on the device. Performance metrics were calculated for each inference, both before and after the warmup process, to compare the results.

	No Warmup	Warmed up
First Inference Time (ms)	18.42 ms	3.44 ms
Speedup After First Inference	4.95x	0.93x
Inference Time Std (ms)	0.584 ms	0.335 ms
Throughput (FPS)	267.88 FPS	271 .62 FPS
Average Inference Time (ms)	3.733 ms	3.682 ms

Looking at this data, we can see a very large difference between the warming up and not warming up in the very first inference (~18ms vs ~3ms). However, the average inference latency is very similar over the course of the 1000 iterations. This indicates that it is just that very first inference on the model that is somewhat slower than others. Due to the process of warming up, we do see a lower standard deviation of inference latencies between the warmed up model vs the non-warmed up model. This is likely due to how large the very first inference without warming up is. The throughput between the two is relatively the same. This indicates that after the cold started model gets going, the performance eventually evens out to the warmed up model.

3) Model Sizes

While optimizing our model architecture for TPU deployment, we observed an interesting outcome regarding model size. The TPU-optimized model is approximately 4M3B, compared to the baseline model's 3MB. This size difference stems from architectural choices made specifically for TPU optimization. The increased size primarily results from the addition of an attention mechanism and the use of TPU-efficient layer dimensions. The attention mechanism, implemented through dense layers with 512 units, adds additional parameters but enables the model to focus on the most relevant features for classification. Additionally, we structured the network with power-of-2 dimensions (512 and 256 units) to align with TPU's optimal processing capabilities, and included BatchNormalization layers to stabilize training. Although these modifications increased the model size by approximately 33%, they represent a calculated trade-off between model size and computational efficiency. The larger model leverages TPU-specific optimizations that can potentially lead to better hardware utilization and improved inference performance, demonstrating how hardware-aware model design sometimes requires balancing competing factors such as model size and execution efficiency.

4) Model Accuracy

During training, our TPU-accelerated model demonstrated strong performance metrics across all datasets. The model achieved 99.57% accuracy on the training set, indicating excellent ability to learn from the training data. The validation accuracy of 90.82% and test accuracy of 90.429% suggest good generalization to unseen data, with minimal overfitting despite the high training accuracy. These metrics were obtained while running the full-precision model on cloud TPUs, leveraging their significant computational capabilities.

Quantization can sometimes result in degraded performance of models, but it is a requirement to deploy a TensorFlow model to the Edge TPU. However, our evaluation on the Edge TPU device yielded impressive results, with the quantized model achieving 89.09% accuracy on the same test dataset. The minimal decrease of approximately 1.4% in test accuracy between the full-precision and quantized versions demonstrates the robustness of our model architecture and the effectiveness of our quantization process.

This performance retention is particularly noteworthy as it validates our hardware-aware design choices. The attention mechanism and TPU-optimized layer dimensions not only contributed to the model's classification ability but also proved resilient to quantization effects. The small accuracy drop suggests that our model's learned features and attention weights remained meaningful even after quantization.

V. CONCLUSION

This project demonstrated the significant potential of TPU architecture in both training and edge deployment scenarios, while showcasing advanced techniques in deep learning optimization and hardware-aware model design. Through comprehensive implementation and analysis, we explored the intricacies of building efficient, parallel data pipelines, leveraging TPU capabilities in cloud environments, and successfully deploying optimized models to edge devices.

Our implementation journey began with the development of a sophisticated data pipeline that maximized TPU utilization through careful optimization of batch sizes, memory management, and parallel processing techniques. By implementing TPU-specific optimizations such as power-of-two dimension sizing and efficient memory access patterns, we created a training environment that could fully leverage the TPU's matrix processing capabilities. The integration of TensorFlow with TPU architecture required careful consideration of data types, tensor shapes, and computational graphs, leading to a robust and efficient training framework.

The model architecture itself represents a thoughtful balance between computational efficiency and classification accuracy. By incorporating an attention mechanism into our fine-tuned MobileNetV2 base model, we demonstrated how advanced deep learning techniques can enhance even relatively straightforward computer vision tasks. The attention layers enable the model to dynamically focus on the most relevant features of each flower image, contributing to our impressive 90% accuracy on the test dataset. This success challenges the notion that attention mechanisms are only valuable in more complex applications, showing their potential benefit in focused classification tasks.

Our performance analysis revealed interesting insights about the trade-offs involved in hardware-aware model design. While our TPU-optimized model showed a slight increase in size compared to the baseline version (4MB versus 3MB), this represented a calculated compromise between model size and computational efficiency. The additional parameters introduced by the attention mechanism and TPU-friendly layer sizes were justified by the improved hardware utilization and inference performance they enabled.

The project culminated in a successful edge deployment, where we implemented comprehensive benchmarking to measure various performance metrics including inference time, throughput, and hardware utilization. Our analysis of warm-up effects and inference timing patterns provided valuable insights into the practical considerations of edge deployment, while our detailed performance monitoring helped validate the effectiveness of our TPU-specific optimizations.

This implementation demonstrates the importance of hardware-aware model design and optimization in modern machine learning applications. The project not only achieved its technical objectives but also provided practical insights into the synergy between model architecture, hardware capabilities, and performance optimization in real-world machine learning deployments.

VI. Related Works

There are a few existing projects that are related to this project. Mainly the idea of applying attention to image classification, TPU optimizations, and work on implementing parallel processing in a TensorFlow training pipeline. One related work is "CBAM: Convolutional Block Attention Module" which demonstrates how to implement an attention layer into a CNN. This is similar in that attention layers are used to enhance a computer vision task and/or CNN, but the implementation is slightly different. The attention mechanism used here only uses channel attention, whereas CBAM utilizes both channel and spatial attention. There are many related works on TensorFlow parallel processing in the TensorFlow documentation that were referenced throughout this project. Additionally, Google docs on TPUs and Coral Dev Board docs on the Edge TPU and examples in there were leveraged extensively.